

Универзитет у Београду
Факултет организационих наука
Катедра за софтверско инжењерство



Семинарски рад из предмета
Напредне јава технологије

Веб апликација за резервацију хотелских соба у Spring Boot и React окружењу

Ментор: проф. др Илија Антовић

Студент: Лука Грчић

Број индекса: 2022/0252

Београд, 2025.

Садржај

1. Вербални опис.....	3
Основни циљеви.....	3
Корисничке улоге и функционалности.....	3
Архитектура и технологије	4
Закључак	4
2. Технологије.....	5
2.1. Spring Boot.....	5
2.2. Spring Boot Web	5
2.3. Spring Data JPA	6
2.4. React.js	6
2.5. Maven.....	7
2.6. MySQL	7
3. Инсталација и општи опис структуре клијентског дела апликације.....	8
Инсталација и покретање пројекта	8
Коршћене технологије.....	9
Структура пројекта.....	9
Опис главних компоненти	11
4. Случајеви коришћења	13
5. Приказ функционалности апликације кроз код клијентског дела апликације	27
6. Структура пројекта	56
7. Архитектура пројекта	58
8. Закључак.....	58

1. Вербални опис

Пројекат „**hotelReservations**“ представља савремену веб апликацију за управљање системом резервација соба у хотелу. Развијен је по клијент–сервер архитектури, при чему је позадински део (backend) имплементиран у **Java Spring Boot** оквиру, а кориснички интерфејс (frontend) реализован помоћу **React.js**.

Главни циљ система је дигитализација и аутоматизација процеса изнајмљивања хотелских соба, уз централно управљање смештајним јединицама, гостима, запосленима и резервацијама.

Основни циљеви:

- **За администраторе:** омогућити ефикасно управљање собама, типовима соба, гостима и резервацијама кроз једноставан административни интерфејс.
- **Технички циљ:** обезбедити стабилан, сигуран и скалабилан систем, изграђен на принципима модерне софтверске архитектуре (**REST API**, сервисна слојевитост, реактиван интерфејс).

Корисничке улоге и функционалности

Систем подржава једног корисничког типа – **администратора**, који управља целокупним системом резервација.

Основне функционалности администратора:

1. **Управљање собама (CRUD):**
 - Филтрирање и претрага по броју собе и по цени по ноћи.
 2. **Управљање гостима (CRUD):**
 - Регистрација гостију који бораве у хотелу (ЈМБГ, име, презиме, е-пошта, телефон, град).
 - Филтрирање и претрага гостију.
 - Брисање постојећих гостију.
 - Ажурирање постојећих гостију.
 3. **Управљање резервацијама:**
 - Креирање нових резервација са дефинисаним датумима доласка и одласка.
 - Брисање постојећих резервација.
 - Претрага и филтрирање резервација.
-

Архитектура и технологије

Backend:

- Језик: **Java**
- Оквир: **Spring Boot**
- База података: **MySQL**
- ORM: **Hibernate** (преко Spring Data JPA)
- Архитектура: **REST API**

Frontend:

- Језик: **TypeScript**
 - Оквир: **React.js**
 - Структура: компонентно оријентисана архитектура са сервисима (*api.js*) за комуникацију са backend-ом
 - UI: чист и минималистички дизајн, табеларни приказ података и интерактивне форме
-

Закључак

„**hotelReservations**“ представља комплетан систем за управљање хотелским резервацијама, који покрива све оперативне процесе хотела — од евиденције соба и гостију до контроле и управљања резервацијама.

Захваљујући стабилној архитектури и јасно дефинисаним валидационим правилима, апликација је поуздана, једноставна за употребу и спремна за даља проширења, као што су:

- систем за обраду плаћања,
- аутентификација више корисничких улога (нпр. рецепционер, менаџер, гост),
- напредно извештавање и статистика попуњености хотела.

2. Технологије

2.1. Spring Boot

Spring Boot је оквир заснован на Јави који омогућава брз и једноставан развој backend апликација.

У оквиру пројекта **hotelReservations**, Spring Boot се користи за имплементацију **REST API** сервиса који повезује React frontend са **MySQL** базом података.

Омогућава аутоматску конфигурацију, минималан број XML подешавања и модуларни приступ кроз „starter“ зависности. У комбинацији са **Spring Data JPA**, овај оквир управља ентитетима као што су *Room*, *Guest*, *Reservation* и *Employee*.

У пројекту се користи за:

- креирање REST контролера за CRUD операције (собе, гости, резервације);
- повезивање слојева контролера, сервиса и репозиторијума;
- аутоматску конфигурацију сервера (уграђени Tomcat).

2.2. Spring Boot Web

Spring Boot Web модул омогућава креирање веб апликација и REST API сервиса.

Он обрађује **HTTP захтеве** и шаље **JSON одговоре** ка клијенту.

У систему **hotelReservations**, овај модул се користи за:

- дефинисање REST контролера као што су *RoomController*, *GuestController* и *ReservationController*;
- руковање HTTP методама (**GET**, **POST**, **PUT**, **DELETE**) за управљање подацима;
- комуникацију између React клијента и Spring Boot сервера;
- локално покретање апликације путем уграђеног **Tomcat** сервера без ручног деплоја.

2.3. Spring Data JPA

Spring Data JPA представља део Spring екосистема који поједностављује рад са базом података кроз **ORM (Object-Relational Mapping)**. Он омогућава рад са ентитетима без класичних SQL упита.

У пројекту **hotelReservations**, овај модул:

- управља ентитетима као што су *Room*, *RoomType*, *Guest*, *Reservation*, *Employee*, *City*;
- преко интерфејса (нпр. *RoomRepository*, *ReservationRepository*) аутоматски генерише имплементације метода попут `findAll()`, `save()`, `deleteById()`;
- омогућава дефинисање прилагођених упита, као што је `search()` за напредну претрагу;
- обезбеђује интеграцију са **MySQL** базом и аутоматско креирање табела

2.4. React.js

React.js је библиотека за изградњу динамичких корисничких интерфејса заснована на компонентној архитектури.

У овом пројекту је коришћен уместо Angular-а, са фокусом на једноставност, модуларност и брзу реакцију апликације.

React се у пројекту користи за:

- компоненте као што су *RoomsPage*, *ReservationsPage*, *GuestsPage*;
 - формирање интерактивних форми за унос, измену и брисање података;
 - динамичко освежавање табела без поновног учитавања странице
-

2.5. Maven

Maven је алат за управљање зависностима и изградњу Java пројеката. Користи се у **hotelReservations** апликацији за управљање свим библиотекама, плагинима и процесом компилације.

Главна конфигурација се налази у `pom.xml` фајлу, где су дефинисане зависности као што су:

- `spring-boot-starter-web`
- `spring-boot-starter-data-jpa`
- `mysql-connector-j`
- `lombok`

Maven обезбеђује:

- једноставно укључивање свих потребних библиотека;
 - изградњу извршне **JAR** датотеке;
 - аутоматско управљање зависностима и њиховим верзијама;
-

2.6. MySQL

MySQL је релациона база података која складишти све податке у систему — собе, типове соба, госте, запослене, градове и резервације.

У пројекту **hotelReservations**, MySQL обезбеђује:

- перзистенцију података кроз ентитете;
 - повезаност табела преко страних кључева (нпр. *гост–град*, *соба–тип собе*, *резервација–соба*);
 - спровођење правила целокупности података, укључујући **ON DELETE CASCADE** за аутоматско брисање зависних података.
-

Уз комбинацију ових технологија, **hotelReservations** обезбеђује поуздан, брз и ефикасан систем за управљање хотелским резервацијама, са потпуном интеграцијом између backend-а и frontend-а.

3. Инсталација и општи опис структуре клијентског дела апликације

Инсталација и покретање пројекта

Пројекат „hotel-react-app-main” развијен је коришћењем React библиотеке са TypeScript језиком и Vite алатком.

1. Креирање пројекта:

```
npm create vite@latest hotel-react-app-main -- --template react-ts
```

(Ова команда креира нови React пројекат са TypeScript подршком.)

2. Улазак у фолдер пројекта:

```
cd hotel-react-app-main
```

3. Инсталација основних зависности:

```
npm install
```

(Овом командом се инсталирају све потребне библиотеке које су наведене у package.json фајлу.)

4. Инсталација додатних библиотека:

```
npm install bootstrap  
npm install react-router-dom
```

5. Покретање пројекта:

```
npm run dev
```

(Апликација се подиже на локалном серверу.)

Након ових корака, апликација је покренута на локалној адреси (на пример <http://localhost:5173/>). Vite је омогућио брзо развојно окружење са аутоматским освежавањем при свакој измени у коду.

Коршћене технологије

- React — за развој компонентног интерфејса.
- TypeScript — за статичку типизацију и поузданији код.
- Bootstrap — за брзо и ефикасно стилизовање елемената.
- React Router — за управљање навигацијом унутар апликације.

Структура пројекта

```
src/
├──
├── api/
│   └── http.ts           # Centralizovani HTTP servis
├──
├── components/
│   ├── TopBar.tsx        # Navigaciona traka sa logout funkcijom
│   ├── GuestInfoModal.tsx # Modal sa detaljima gosta i mogućnošću izmene
│   ├── NewGuestModal.tsx  # Modal za dodavanje novog gosta
│   ├── ReservationInfoModal.tsx # Modal sa detaljima rezervacije
│   └── NewReservationModal.tsx # Modal za kreiranje nove rezervacije
├──
├── pages/
│   ├── LoginPage.tsx      # Prijava korisnika
│   ├── GuestsPage.tsx     # Stranica sa listom gostiju
│   ├── RoomsPage.tsx      # Stranica sa listom soba
│   └── ReservationsPage.tsx # Stranica sa listom rezervacija
├──
├── services/
│   ├── auth.ts           # Servis za autentifikaciju (login, token)
│   └── reservations.ts    # Servis za sve API pozive (gosti, sobe, rezervacije)
├──
├── App.tsx               # Glavna aplikacija i definisane rute
├── main.tsx              # Ulazna tačka aplikacije
└── index.html            # Osnovna HTML stranica sa <div id="root">
```

```
✓ HOTEL-REACT-APP-MAIN
  > node_modules
  > public
  ✓ src
    ✓ api
      TS http.ts
    > assets
    ✓ components
      ✨ GuestInfoModal.tsx
      ✨ NewGuestModal.tsx
      ✨ NewReservationModal.tsx
      ✨ ReservationInfoModal.tsx
      ✨ TopBar.tsx
    ✓ pages
      ✨ GuestsPage.tsx
      ✨ LoginPage.tsx
      ✨ ReservationsPage.tsx
      ✨ RoomsPage.tsx
    ✓ services
      TS auth.ts
      TS reservations.ts
      # App.css
      ✨ App.tsx
      # index.css
      ✨ main.tsx
    ✨ .gitignore
    ✨ eslint.config.js
    <> index.html
    {} package-lock.json
    {} package.json
    ⓘ README.md
    {} tsconfig.app.json
    TS tsconfig.json
    {} tsconfig.node.json
    ⚡ vite.config.ts
```

Опис главних компоненти

App.tsx — Главна компонента која дефинише све руте у апликацији користећи React Router. Компонента RequireAuth проверава да ли је корисник пријављен пре него што му дозволи приступ страницама као што су ReservationsPage, GuestsPage и RoomsPage.

main.tsx — Улазна тачка апликације која рендерује App компоненту у DOM преко ReactDOM.createRoot. Овде се укључује и Bootstrap библиотека како би се омогућило коришћење стилизованих компоненти.

LoginPage.tsx — Компонента која омогућава кориснику пријаву у систем. Корисник уноси корисничко име и лозинку, а по успешној пријави JWT токен се чува у localStorage-у. Након тога, корисник се преусмерава на страницу резервација.

ReservationsPage.tsx — Страница која приказује све постојеће резервације у табеларном приказу. Омогућена је претрага, преглед детаља, брисање резервација и отварање модала за нову резервацију. Користи useEffect за иницијално учитавање података и useState за управљање стањем странице.

RoomsPage.tsx — Страница која приказује све собе у систему. Омогућена је претрага по броју, типу или спрату. Функција fmtPrice форматира цену у валуту (евро) ради боље читљивости.

GuestsPage.tsx — Страница за управљање подацима о гостима. Приказује све госте и омогућава додавање нових, као и измену постојећих.

TopBar.tsx — Горња навигациона трака која приказује корисничко име тренутно пријављеног радника, омогућава одјаву (logout) и садржи падајући мени за прелазак на различите странице апликације.

ReservationInfoModal.tsx — Модални дијалог који приказује детаље о појединачној резервацији. Док је модал отворен, позадински скрол је онемогућен. Приказује податке као што су соба, датум, гост и цена.

NewReservationModal.tsx — Компонента за додавање нове резервације. Омогућава претрагу гостију и соба са 'debounce' логиком да би се избегли сувишни API позиви. Рачуна број ноћења и укупну цену боравка на основу изабраних датума. Садржи валидацију и приказује резиме цене пре слања података.

NewGuestModal.tsx — Компонента за додавање новог госта у систем. Садржи више поља за унос података (ЈМБГ, име, презиме, мејл, град...) и свеобухватну валидацију улазних података пре чувања. Приликом отварања модала аутоматски се учитава листа градова.

GuestInfoModal.tsx — Компонента која приказује податке о изабраном госту и омогућава њихову измену. Укључује прекидач за прелаз у режим уређивања (edit mode). Користи useМемо за ефикасно рендеровање падајуће листе градова.

auth.ts — Сервис за аутентификацију који садржи функције за пријаву, чување, преузимање и брисање JWT токена. Функција saveAuth чува податке у localStorage, док logout брише токен и податке о кориснику.

http.ts — Фајл који централизује HTTP комуникацију са сервером. Функција http<T> шаље захтеве користећи Fetch API и аутоматски додаје 'Authorization' хедер ако постоји токен. Такође обрађује одговоре и баца грешке ако сервер врати неуспешан статус.

reservations.ts — Сервис који централизује све API позиве везане за резервације, госте, собе и градове. Користи помоћну функцију http за комуникацију са backend-ом. Дефинише све TypeScript типове података (Guest, Room, City, Reservation итд.) који описују структуру одговора и захтева ка серверу. Омогућава CRUD операције — претрагу, листање, додавање, ажурирање и брисање ентитета. Користи TypeScript генерике како би обезбедио типизацију одговора са API-ја, што повећава сигурност и читљивост кода. Ово представља централно место за одржавање логике комуникације између фронт-енда и серверског дела апликације.

4. Случајеви коришћења

За следеће случајеве коришћења ћемо дати детаљан опис:

СК1	Пријава запосленог
СК2	Претрага госта
СК3	Креирање госта
СК4	Измена госта
СК5	Брисање госта
СК6	Претрага резервације
СК7	Креирање резервације
СК8	Брисање резервације
СК9	Претрага соба

СК1 – Пријава запосленог

Назив СК:

Пријава запосленог

Актори СК:

Запослени

Учесници СК:

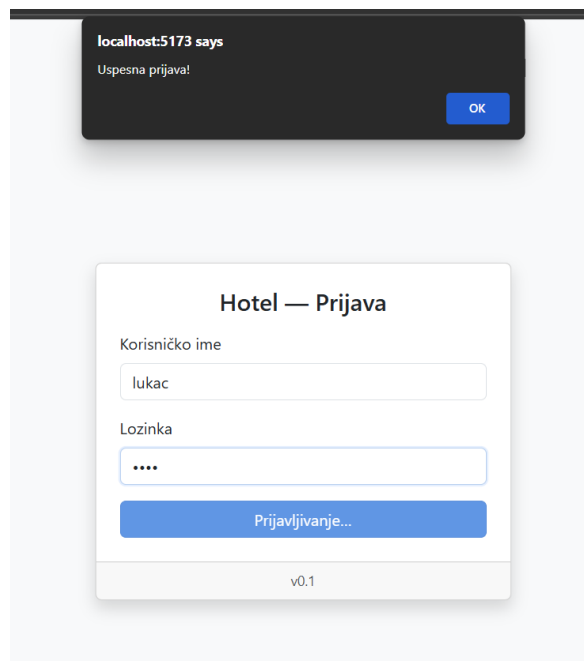
Запослени, кориснички интерфејс (React апликација), систем (Spring Boot сервер)

Предуслови:

Клијентска и серверска апликација су покренуте. Запослени је регистрован у систему.

Основни сценарио СК:

1. Запослени уноси корисничко име и лозинку. (АПУСО)
2. Запослени позива систем за пријаву. (АПСО)
3. Систем проверава податке у бази. (СО)
4. Систем приказује поруку: „Успешна пријава!“ (ИА)



Алтернативна сценарија:

4.1. Ако су подаци невалидни, систем приказује поруку: „Prijava nije uspela“ (ИА)

The image shows a login form titled "Hotel — Prijava". At the top, there is a red error message box that says "Prijava nije uspela". Below this, there are two input fields: "Korisničko ime" (Username) with the value "lukac" and "Lozinka" (Password) with masked characters "....". A blue button labeled "Prijavi se" (Login) is positioned below the password field. At the bottom of the form, the version "v0.1" is displayed.

Hotel — Prijava

Prijava nije uspela

Korisničko ime

lukac

Lozinka

....

Prijavi se

v0.1

СК2 – Претрага госта

Назив СК:
Претрага госта

Актори СК:
Запослени

Учесници СК:
Запослени, интерфејс, систем

Предуслови:
Клијент и сервер су покренути, а запослени је пријављен.

Основни сценарио СК:

1. Запослени уноси параметар претраге. (АПУСО)
2. Запослени позива систем да изврши претрагу. (АПСО)
3. Систем претражује госте у бази података. (СО)
4. Систем приказује резултате у табели. (ИА)

Gosti							Mil	Traži	Novi gost
Šifra gosa	JMBG	Ime	Prezime	Email	Telefon	Akcije			
22	9012345678901	Sara	Milanović	sara.milanovic@email.com	+381641901234	Info	Obriši		
24	1122334455667	Milica	Todorović	milica.todorovic@email.com	+381642123456	Info	Obriši		

СКЗ – Креирање госта
Назив СК:
Креирање госта

Актори СК:
Запослени

Учесници СК:
Запослени, интерфејс, систем

Предуслови:
Сервер и клијент су покренути, а запослени је пријављен.

Основни сценарио СК:

1. Запослени отвара форму за додавање госта. (АПСО)
2. Систем приказује форму са пољима. (ИА)
3. Запослени уноси податке. (АПУСО)
4. Запослени позива систем да сачува госта. (АПСО)
5. Систем чува податке у базу. (СО)
6. Систем приказује поруку: „Гост је сачуван!“ (ИА)

The screenshot displays a web application for hotel management. At the top, there's a header with 'Hotel • Frontdesk' and a user notification 'localhost:5173 says Gost je sacuvan!' with an 'OK' button. Below the header is a table of guests with columns for ID, JMBG, Name, Surname, Email, and Phone. The table contains five rows of data. To the right of the table are buttons for 'Traži' and 'Novi gost'. A modal window titled 'Novi gost' is open in the foreground, containing form fields for JMBG, Email, Ime (First Name), Prezime (Surname), Telefon (Phone), and Grad (City). The 'Novi gost' form has 'Otkazi' and 'Čuvam...' buttons at the bottom.

Šifra gosta	JMBG	Ime	Prezime	Email	Telefon	Akcije
14	2234567890123	Marko	Petrović	marko.petrovic@email.com	+381641234567	Info Obriši
15	2345678901234	Ana	Jovanović	ana.jovanovic@email.com	+381641345678	Info Obriši
17	3456789012345	Nikola	Nikolić	nikola.nikolic@email.com	+381641456789	Info Obriši
18	5678901234567	Jovana	Savić	jovana.savic@email.com	+381641567890	Info Obriši
19						Info Obriši
20						Info Obriši
21						Info Obriši
22						Info Obriši
23						Info Obriši
24						Info Obriši
26						Info Obriši

Алтернативна сценарија:

6.1. У случају грешке исписаће се одговарајућа порука. (ИА)

The screenshot shows a web application interface for a hotel's frontdesk. A dark modal message box at the top center displays the text "localhost:5173 says" and "JMBG mora imati tačno 13 cifara!" (JMBG must have exactly 13 digits!). Below this, a table lists existing guests with columns for ID, JMBG, Name, Email, and Phone. A "Novi gost" (New guest) form is open in the foreground, containing fields for JMBG, Email, Ime (First Name), Prezime (Last Name), Telefon (Phone), and Grad (City). The JMBG field contains "423525", which is the source of the error message. The form also includes "Otkazi" (Cancel) and "Sačuvaj" (Save) buttons.

ID	Šifra gosta	JMBG	Ime	Prezime	Email	Telefon	Grad	Akcije
14	2234567890123	Marko	Petrović	marko.petrovic@email.com	+381641234567	Info	Obrisi	
15	2345678901234	Ana	Jovanović	ana.jovanovic@email.com	+381641345678	Info	Obrisi	
17	3456789012345	Nikola	Nikolić	nikola.nikolic@email.com	+381641456789	Info	Obrisi	
18	5678901234567	Jovana	Savić	jovana.savic@email.com	+381641567890	Info	Obrisi	

Novi gost

JMBG: 423525

Email: dragan.peric@gmail.com

Ime: Dragan

Prezime: Peric

Telefon: +381636896534

Grad: Paracin

Otkazi Sačuvaj

СК4 – Измена госта

Назив СК:

Измена госта

Актери СК:

Запослени

Учесници СК:

Запослени, интерфејс, систем

Предуслови:

Запослени је пријављен и гост постоји у систему.

Основни сценарио СК:

1. Запослени претражује госте. (АПУСО)
2. Систем приказује резултате. (ИА)
3. Запослени одабира госта за измену. (АПУСО)
4. Запослени мења податке. (АПУСО)
5. Запослени позива систем да сачува измене. (АПСО)
6. Систем ажурира податке у бази. (СО)
7. Систем приказује поруку: „Подаци о госту су успешно ажурирани!“ (ИА)

The screenshot displays the 'Hotel • Frontdesk' interface. At the top, there's a search bar labeled 'Pretraga (ime, prezime, JMBG, €)' with a 'Traži' button and a 'Novi gost' button. Below this is a table of guests with columns: Šifra gosta, JMBG, Ime, Prezime, Email, Telefon, and Akcije. The table lists four guests. A modal window titled 'Detalji gosta' is open, showing the details for the first guest (Marko Petrović). The modal has input fields for JMBG, Email, Ime, Prezime, Telefon, and Grad, with a dropdown for the city. At the bottom of the modal are buttons for 'Uredi', 'Zatvori', and 'Sačuvaj izmene'.

Šifra gosta	JMBG	Ime	Prezime	Email	Telefon	Akcije
14	2234567890123	Marko	Petrović	marko.petrovic@email.com	+381641234567	Info Obriši
15	2345678901234	Ana	Jovanović	ana.jovanovic@email.com	+381641345678	Info Obriši
17	3456789012345	Nikola	Nikolić	nikola.nikolic@email.com	+381641456789	Info Obriši
18	5678901234567	Jovana	Savić	jovana.savic@email.com	+381641567890	Info Obriši

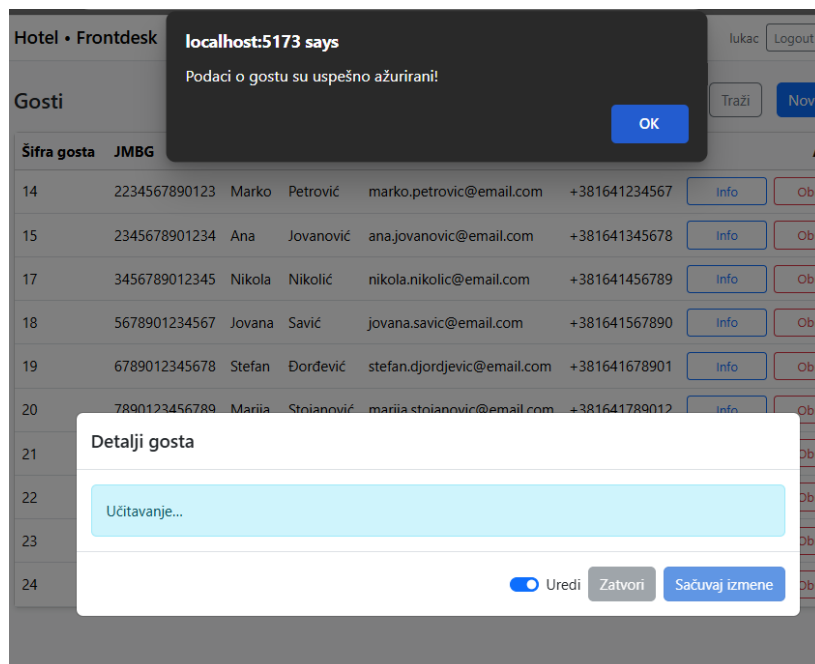
Detalji gosta

JMBG: Email:

Ime: Prezime:

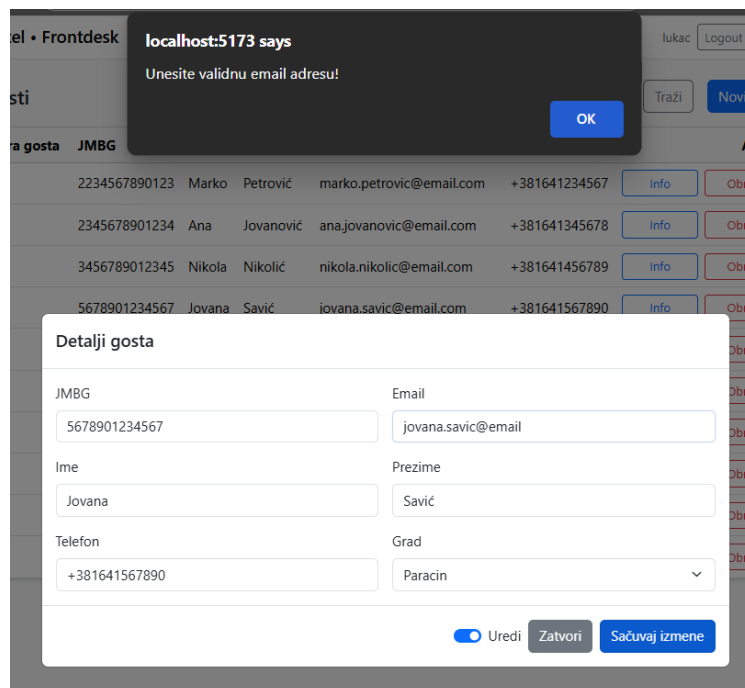
Telefon: Grad:

☒ Uredi



Алтернативна сценарија:

7.1. У случају грешке исписаће се одговарајућа порука.(ИА)



СК5 – Брисање госта

Назив СК:

Брисање госта

Актери СК:

Запослени

Учесници СК:

Запослени, интерфејс, систем

Предуслови:

Запослени је пријављен.

Основни сценарио СК:

1. Запослени претражује госте. (АПУСО)
2. Систем приказује резултате. (ИА)
3. Запослени одабира госта и позива систем за брисање. (АПСО)
4. Систем брише госта из базе. (СО)
5. Систем приказује поруку: „Гост је успешно обрисан!“ (ИА)

Hotel • Frontdesk

localhost:5173 says
Gost je uspešno obrisan!
OK

lukac Logout

Traži Novi gost

Gosti

Šifra gosa	JMBG						Akcije
14	2234567890123	Marko	Petrović	marko.petrovic@email.com	+381641234567	Info	Obriši
15	2345678901234	Ana	Jovanović	ana.jovanovic@email.com	+381641345678	Info	Obriši
17	3456789012345	Nikola	Nikolić	nikola.nikolic@email.com	+381641456789	Info	Obriši
18	5678901234567	Jovana	Savić	jovana.savic@email.com	+381641567890	Info	Obriši
19	6789012345678	Stefan	Đorđević	stefan.djordjevic@email.com	+381641678901	Info	Obriši
20	7890123456789	Marija	Stojanović	marija.stojanovic@email.com	+381641789012	Info	Obriši
21	8901234567890	Ivan	Pavlović	ivan.pavlovic@email.com	+381641890123	Info	Obriši
22	9012345678901	Sara	Milanović	sara.milanovic@email.com	+381641901234	Info	Obriši
23	0123456789012	Luka	Ristić	luka.ristic@email.com	+381642012345	Info	Obriši
24	1122334455667	Milica	Todorović	milica.todorovic@email.com	+381642123456	Info	Obriši
25	9812873476503	Dragan	Perić	dragan.peric@gmail.co	+381639046936	Info	Obriši

Алтернативна сценарија:

5.1. Ако за госта постоје резервације исписује се порука “Za navedenog gosta postoje rezervacije, nije moguće obrisati gosta.” (ИА)

Hotel • Frontdesk

localhost:5173 says
Error: Za navedenog gosta postoje rezervacije, nije moguće obrisati gosta.
OK

lukac Logout

Traži Novi gost

Gosti

Šifra gosta	JMBG						Akcije
14	2234567890123	Marko	Petrović	marko.petrovic@email.com	+381641234567	Info	Obrisi
15	2345678901234	Ana	Jovanović	ana.jovanovic@email.com	+381641345678	Info	Obrisi
17	3456789012345	Nikola	Nikolić	nikola.nikolic@email.com	+381641456789	Info	Obrisi
18	5678901234567	Jovana	Savić	jovana.savic@email.com	+381641567890	Info	Obrisi
19	6789012345678	Stefan	Đorđević	stefan.djordjevic@email.com	+381641678901	Info	Obrisi
20	7890123456789	Marija	Stojanović	marija.stojanovic@email.com	+381641789012	Info	Obrisi
21	8901234567890	Ivan	Pavlović	ivan.pavlovic@email.com	+381641890123	Info	Obrisi
22	9012345678901	Sara	Milanović	sara.milanovic@email.com	+381641901234	Info	Obrisi
23	0123456789012	Luka	Ristić	luka.ristic@email.com	+381642012345	Info	Obrisi
24	1122334455667	Milica	Todorović	milica.todorovic@email.com	+381642123456	Info	Obrisi

СК6 – Претрага резервације

Назив СК:
Претрага резервације

Актори СК:
Запослени

Учесници СК:
Запослени, интерфејс, систем

Предуслови:
Клијент и сервер су покренути, а запослени је пријављен.

Основни сценарио СК:

1. Запослени уноси параметар претраге. (АПУСО)
2. Запослени позива систем да изврши претрагу. (АПСО)
3. Систем претражује резервације у бази података. (СО)
4. Систем приказује резултате у табели. (ИА)

Rezervacije					
		mil		Traži	Nova rezervacija
Šifra rezervacije	Soba	Datum od	Datum do	Gost	Akcije
7	302	2025-10-19	2025-11-06	Sara Milanović	Info Obriši
8	202	2025-10-19	2025-10-23	Milica Todorović	Info Obriši
9	301	2025-10-29	2025-10-30	Sara Milanović	Info Obriši

СК7 – Креирање резервације

Назив СК:

Креирање резервације

Актори СК:

Запослени

Учесници СК:

Запослени, интерфејс, систем

Предуслови:

Сервер и клијент су покренути, а запослени је пријављен.

Основни сценарио СК:

1. Запослени отвара форму за додавање резервације. (АПСО)
2. Систем приказује форму са пољима. (ИА)
3. Запослени уноси податке. (АПУСО)
4. Запослени позива систем да сачува резервацију. (АПСО)
5. Систем чува податке у базу. (СО)
6. Систем приказује поруку: „Резервација је сачувана!“ (ИА)

The screenshot displays a web application for hotel reservations. At the top, a dark notification banner from 'localhost:5173' states 'Rezervacija je sacuvana!' (Reservation is saved!) with an 'OK' button. Below this, a modal window titled 'Nova rezervacija' is open. The form contains the following fields and options:

- Gost:** A text field with 'Dragan Peric — 4235257834765' and a sub-label 'Izabran: Dragan Peric (ID: 26)'.
- Soba:** A text field with 'Soba 301 (sprat 3) — Suite' and a sub-label 'Izabrana: soba 301 (ID: 5), Suite'.
- Datum od:** A date picker set to '10/29/2025'.
- Datum do:** A date picker set to '10/31/2025'.
- Doručak uključen:** A checked checkbox.
- Napomena:** An empty text area.
- Summary:** A grey box at the bottom left shows 'Noćenja: 2' and 'Cena po noći: €150.00'. A grey box at the bottom right shows 'Ukupno: €300.00'.

At the bottom right of the modal, there are two buttons: 'Otkazi' (Cancel) and 'Čuvam...' (Save).

СК8 – Брисање резервације

Назив СК:

Брисање резервације

Актори СК:

Запослени

Учесници СК:

Запослени, интерфејс, систем

Предуслови:

Запослени је пријављен.

Основни сценарио СК:

1. Запослени претражује резервације. (АПУСО)
2. Систем приказује резултате. (ИА)
3. Запослени одабира резервацију и позива систем за брисање. (АПСО)
4. Систем брише резервацију из базе. (СО)
5. Систем приказује поруку: „Резервација је успешно обрисана!“ (ИА)

The screenshot shows a web application interface for a hotel's frontdesk. At the top, there's a header with "Hotel • Frontdesk" on the left, a user profile "lukac" with a "Logout" button on the right, and a hamburger menu icon. A dark modal box is centered on the screen, displaying the message "localhost:5173 says" and "Rezervacija je uspešno obrisana!" with an "OK" button. Below the modal, the "Rezervacije" section contains a table of reservations. The table has columns for reservation ID, room number, check-in date, check-out date, guest name, and actions. There are five rows of data. To the right of the table, there's a "Nova rezervacija" button and an "Akcije" label.

Šifra rezervacije					Akcije
7	302	2025-10-19	2025-11-06	Sara Milanović	Info Obriši
8	202	2025-10-19	2025-10-23	Milica Todorović	Info Obriši
9	301	2025-10-29	2025-10-30	Sara Milanović	Info Obriši
10	102	2025-10-20	2025-10-29	Marko Petrović	Info Obriši
11	301	2025-10-29	2025-10-31	Dragan Peric	Info Obriši

СК9 – Претрага соба

Назив СК:
Претрага соба

Актори СК:
Запослени

Учесници СК:
Запослени, интерфејс, систем

Предуслови:
Клијент и сервер су покренути, а запослени је пријављен.

Основни сценарио СК:

1. Запослени уноси параметар претраге. (АПУСО)
2. Запослени позива систем да изврши претрагу. (АПСО)
3. Систем претражује собе у бази података. (СО)
4. Систем приказује резултате у табели. (ИА)

Smeštajne jedinice				30	Traži
Sifra sobe	Broj sobe	Sprat	Kategorija	Cena za noc	
5	301	3	Suite	€150.00	
6	302	3	Suite	€150.00	

5. Приказ функционалности апликације кроз код клијентског дела апликације

СК1 – Пријава запосленог

Улазна тачка — страница LoginPage.tsx

Рута за ову компоненту дефинисана је у App.tsx као:

```
{/* Login ruta – jedina koja nije zaštićena jer je javna */}  
<Route path="/" element={<LoginPage />} />
```

То значи да је ово почетна страница која се прва приказује када се апликација отвори.

Логика компоненте LoginPage

Главни део функционалности се налази унутар функције onSubmit():

```
const onSubmit = async (e: FormEvent) => {  
  e.preventDefault() // sprečava podrazumevano osvežavanje stranice nakon submit-a  
  setError(null) // briše prethodnu grešku ako postoji  
  setLoading(true) // prikazuje stanje učitavanja  
  
  try {  
    // Poziva login servis (HTTP POST ka /auth/login)  
    // login vraća objekat sa tokenom, korisničkim imenom i ID-jem.  
    const { token, username: u, id: uid } = await login(username, password)  
  
    // Čuvamo podatke o korisniku u localStorage  
    // Ovaj korak održava sesiju aktivnom i nakon refresh-a stranice  
    saveAuth(token, u, uid)  
  
    // Resetujemo lozinku radi sigurnosti (ne čuvamo je u memoriji)  
    setPassword('')  
  
    // Navigacija korisnika na početnu stranicu rezervacija  
    navigate('/reservations')  
  
    // Jednostavan vizuelni feedback korisniku  
    alert('Uspesna prijava!')  
  } catch (err: any) {  
    // Ako dođe do greške (npr. pogrešni kredencijali ili greška servera)  
    // ispisuje se jednostavna poruka korisniku  
    setError('Prijava nije uspela')  
  } finally {  
    // finally blok se uvek izvršava – resetuje stanje dugmeta  
    setLoading(false)  
  }  
}
```

Корисник попуњава форму са пољима username и password.

Када кликне на „Пријави се“, позива се onSubmit().

Функција блокира подразумевано освежавање странице (preventDefault()).

Поставља стање *учитавања* (setLoading(true)).

Позива функцију login() из сервиса auth.ts.

Позив ка серверу — сервис auth.ts

У овом сервису је дефинисана функција за комуникацију са backend-ом:

```
export async function login(username: string, password: string): Promise<LoginResponse> {
  // Funkcija šalje POST zahtev na /auth/login sa korisničkim kredencijalima.
  // Backend proverava validnost korisnika i vraća JWT token i osnovne podatke.
  return http<LoginResponse>('/auth/login', {
    method: 'POST', // POST jer se šalju poverljivi podaci
    body: JSON.stringify({ username, password }), // Telo zahteva se šalje kao JSON string
  })
}
```

Користи се помоћни модул http.ts са подешеним baseUrl-ом.

Шаље се **POST** захтев на руту auth/login са подацима у телу

Backend враћа одговор

Након успешног одговора, позива се функција saveAuth() из auth.ts

```
export function saveAuth(token: string, username: string, userId: string) {
  // Čuva podatke o sesiji u localStorage-u browsera.
  // Ovo omogućava da korisnik ostane prijavljen i posle osvežavanja stranice.
  localStorage.setItem('authToken', token) // JWT token
  localStorage.setItem('authUser', username) // korisničko ime
  localStorage.setItem('authUserId', userId) // ID korisnika
}
```

token се користи за аутентификацију у свим следећим API позивима.

username и **userId** се користе у интерфејсу (нпр. приказ у TopBar-y).

Ово омогућава да пријава остане активна и након освежавања странице.

Ако је пријава успешна корисник се аутоматски преусмерава на страницу са резервацијама (ReservationsPage).

```
// Navigacija korisnika na početnu stranicu rezervacija
navigate('/reservations')
```

На тој рути, у App.tsx, стоји **RequireAuth** заштита:

```
/* Zaštićene rute – RequireAuth proverava da li postoji token */
<Route
  path="/reservations"
  element={
    <RequireAuth>
      <ReservationsPage />
    </RequireAuth>
  }
/>
```

Она проверава да ли у localStorage постоји токен.

```
function RequireAuth({ children }: { children: JSX.Element }) {
  const token = getToken() // čitamo token iz localStorage-a pomoću pomoćne funkcije iz auth servisa
  // Ako token ne postoji, korisnik nije prijavljen → redirectujemo ga na "/" (login stranicu)
  if (!token) return <Navigate to="/" replace />
  // Ako postoji token, vraćamo dete-komponentu (npr. ReservationsPage)
  return children
}
```

Ако токен не постоји (нпр. ако се корисник није пријавио или се одјавио), аутоматски се враћа на **LoginPage**.

CK2 – Претрага госта

Улазна тачка — страница GuestsPage.tsx

Ово је главна страница за рад са гостима. Рута се дефинише у App.tsx:

```
<Route
  path="/guests"
  element={
    <RequireAuth>
      <GuestsPage />
    </RequireAuth>
  }
/>
```

Страницу могу отворити само пријављени корисници.

Главне state променљиве у компоненти

```
const [data, setData] = useState<Guest[]>([]) // lista gostiju prikazana u tabeli
const [loading, setLoading] = useState(true) // indikator inicijalnog učitavanja
const [error, setError] = useState<string | null>(null) // poruka o grešci
const [query, setQuery] = useState('') // tekst za pretragu
const [searching, setSearching] = useState(false) // indikator aktivne pretrage
```

Компонента има и функцију **refreshList(term)** која управља свим позивима ка API-ју — било да се ради о иницијалном читавању или претрази

Кључна функција — refreshList(term: string)

```
// Ova pomoćna funkcija učitava listu gostiju.
// Ako je "term" prazan → dohvata sve; inače koristi endpoint za pretragu.
const refreshList = async (term: string) => {
  setSearching(true)
  setError(null)
  try {
    const t = term.trim() // uklanja prazne razmake iz pretrage
    if (!t) {
      // Ako nema pretrage – GET /guests/findAll
      setData(await listGuests())
    } else {
      // Ako postoji upit – GET /guests/search?query=...
      setData(await searchGuests(t))
    }
  } catch (e: any) {
    // hvata potencijalnu grešku (npr. mrežnu ili backend)
    setError(e?.message ?? 'Učitavanje nije uspelo')
  } finally {
    setSearching(false)
  }
}
```

Стање „претрага у току“ → `searching = true`.

Ако је поље за претрагу празно → позива се `listGuests()` и враћа се комплетна листа.

Ако има унетог текста → позива се `searchGuests(t)` из `reservations.ts` сервиса.

У зависности од одговора, ажурира се `data` — садржај табеле гостију.

Сервисни позиви у `reservations.ts`

```
export async function listGuests(): Promise<Guest[]> {
  // Враћа listu svih gostiju
  // Standardni GET zahtev bez dodatnih parametara
  return http<Guest[]>('/guests/findAll')
}

export async function searchGuests(query: string): Promise<Guest[]> {
  // Trimujemo i proveravamo da li postoji upit
  const q = query?.trim() ?? ''
  // Ako korisnik nije uneo tekst (query je prazan), vraćamo prazan niz – ne šaljemo zahtev
  if (!q) return []
  // Ako upit postoji, formira se URL sa query parametrom
  return http<Guest[]>(`/guests/search?query=${encodeURIComponent(q)}`)
}
```

оба користе заједнички `http` модул из `http.ts`, који додаје заглавља са токеном за аутентификацију.

Догађај покретања претраге — `onSearchSubmit()`

Функција која се активира када корисник кликне на **дугме „Тражи“**:

```
const onSearchSubmit = (e: React.FormEvent) => {
  e.preventDefault() // sprečava reload stranice
  refreshList(query)  // poziva helper koji pretražuje goste
}
```

Она прослеђује тренутни текст из поља `query` у `refreshList()`.

JSX — форма за унос и приказ резултата

```
{/* Forma za pretragu */}
<form className="d-flex ms-auto" onSubmit={onSearchSubmit} role="search">
  <input
    className="form-control me-2"
    style={{ width: 260 }}
    type="search"
    placeholder="Pretraga (ime, prezime, JMBG)"
    value={query}
    onChange={(e) => setQuery(e.target.value)} // ažurira tekst pretrage
  />
  <button className="btn btn-outline-secondary" type="submit" disabled={searching}>
    {searching ? 'Tražim...' : 'Traži'}
  </button>
</form>
```

Свако куцање уноси вредност у query кроз setQuery().

Када се кликне „Тражи“, позива се onSearchSubmit() → refreshList() → API.

Приказ резултата у табели

Када refreshList() постави data, табела се динамички рендерује:

```
{data.map((g) => (
  <tr key={g.id}>
    <td className="text-monospace">{g.id}</td>
    <td>{g.jmbg}</td>
    <td>{g.firstName}</td>
    <td>{g.lastName}</td>
    <td>{g.email}</td>
    <td>{g.phoneNumber}</td>
```

резултати из backend-а се приказују одмах, без освежавања странице.

СКЗ – Креирање госта

Улазна тачка — страница `GuestsPage.tsx`

На врху странице са листом гостију постоји дугме:

```
    {/* Дугме за отварање модала за креирање новог госта */}  
    <button className="btn btn-primary ms-2" onClick={() => setCreateOpen(true)}>  
      Нови гост  
    </button>  
  </div>
```

Када се на њега кликне → променљива `createOpen` постаје `true`, и отвара се **модални прозор** `NewGuestModal`.

```
    {/* Modal za kreiranje novog gosta.  
     * Nakon uspešnog kreiranja → refreshuje prikaz liste. */}  
    <NewGuestModal  
      show={createOpen}  
      onClose={() => setCreateOpen(false)}  
      onCreate={() => refreshList(query)}  
    />  
  </div>
```

show – контролише да ли је модал видљив.

onClose() – затвара модал без акције.

onCreated() – након успешног чувања освежава листу гостију.

Компонента NewGuestModal.tsx

Главна компонента која садржи форму за унос података новог госта

State променљиве:

```
// Čuva listu gradova koju učitavamo iz baze da bi se prikazala u <select> elementu
const [cities, setCities] = useState<City[]>([])
// loading označava da li je u toku API zahtev (učitavanje gradova ili snimanje gosta)
const [loading, setLoading] = useState(false)
// error sadrži tekst greške koji se prikazuje korisniku ako API padne
const [error, setError] = useState<string | null>(null)

// State za formu – svaki input u formi ažurira neko od ovih polja.
// cityId se popunjava automatski nakon što se učitaju gradovi
const [form, setForm] = useState<SaveGuestRequest>({ (property) phoneNumber: string
  jmbg: '', firstName: '', lastName: '', email: '', phoneNumber: '', cityId: 0,
})
```

cities → листа градова за падајући мени (учитава се са backend-a).

form → представља сва поља која корисник попуњава.

Учитавање листе градова при отварању модала

Када се модал отвори (show === true), активира се useEffect:

```
useEffect(() => {
  if (!show) return // ako je modal zatvoren, ne radimo ništa

  setError(null)
  setLoading(true)

  ;(async () => {
    try {
      const cs = await listCities() // GET /cities/findAll
      setCities(cs)
      // Ako API vrati listu, automatski postavi prvi grad kao izabrani
      setForm((f) => ({ ...f, cityId: cs[0]?.cityId ?? 0 }))
    } catch (e: any) {
      // Ako poziv padne, ispiši poruku greške u alert unutar modala
      setError(e?.message ?? 'Neuspešno učitavanje gradova')
    } finally {
      setLoading(false)
    }
  })()
}, [show])
```

Сваки пут кад се модал отвори, апликација позива `listCities()` из `reservations.ts`.

Ако је позив успешан, први град у листи се аутоматски поставља као подразумеван.

Ако не, приказује се порука о грешци.

Попуњавање форме

Свако `input` поље у форми је „controlled component“:

```
<div className="col-sm-6">
  <label className="form-label">Име</label>
  <input
    className="form-control"
    value={form.firstName}
    onChange={(e) => onChange({ firstName: e.target.value })}
  />
</div>
```

Функција `onChange()` ажурира само оно поље које је измењено:

```
// Помоћна функција за ажурирање стања форме.
// Овај приступ (patch објекат) омогућава да менјамо само једно поље без prepisivanja cele forme.
const onChange = (patch: Partial<SaveGuestRequest>) =>
  setForm((f) => ({ ...f, ...patch })))
```

Валидација форме

Пре него што се подаци пошаљу, извршава се функција `validateForm()`:

```
// Validacija forme pre slanja. Koristi alert() umesto inline poruka radi jednostavnosti.
const validateForm = (): boolean => {
  if (!form) {
    alert('Forma nije inicijalizovana!')
    return false
  }

  // Proveravamo da nijedno obavezno polje nije prazno.
  if (!form.jmbg.trim()) { alert('Polje "JMBG" ne sme biti prazno!'); return false }
  if (!form.firstName.trim()) { alert('Polje "Ime" ne sme biti prazno!'); return false }
  if (!form.lastName.trim()) { alert('Polje "Prezime" ne sme biti prazno!'); return false }
  if (!form.email.trim()) { alert('Polje "Email" ne sme biti prazno!'); return false }
  if (!form.phoneNumber.trim()) { alert('Polje "Telefon" ne sme biti prazno!'); return false }
  if (!form.cityId) { alert('Polje "Grad" ne sme biti prazno!'); return false }

  // Email mora biti u validnom formatu (regex proverava osnovnu strukturu)
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
  if (!emailRegex.test(form.email)) {
    alert('Unesite validnu email adresu!')
    return false
  }

  // JMBG mora imati tačno 13 cifara i sadržati samo brojeve
  if (form.jmbg.length !== 13 || !/^\d+$/.test(form.jmbg)) {
    alert('JMBG mora imati tačno 13 cifara!')
    return false
  }

  return true
}
```

Слање форме — функција `submit()`

```
// Poziva se kada korisnik klikne na dugme "Sačuvaj".
// Radi sledeće korake: validira, šalje podatke serveru, prikazuje poruku i zatvara modal.
const submit = async () => {
  // Dodatna provera – ako nisu popunjena sva polja, prekini
  if (!canSubmit) return
  // Validacija forme (npr. format email-a, broj cifara JMBG-a)
  if (!validateForm()) return

  try {
    setLoading(true)
    setError(null)
    await saveGuest(form) // POST /guests/save

    // Ako zahtev prode, obaveštavamo korisnika alertom
    alert('Gost je uspešno sačuvan!')

    // Zatvaramo modal i obaveštavamo roditelja da se lista osveži
    onClose()
    onCreated()
  } catch (e: any) {
    // Ako se desi greška (npr. backend vrati 400 ili 500), prikazujemo poruku
    setError(e?.message ?? 'Snimanje gosta nije uspelo')
  } finally {
    setLoading(false)
  }
}
```

Врши се валидација.

Ако су подаци исправни → позива се `saveGuest()` из `reservations.ts`.

Backend враћа потврду о успешно креираном госту.

Приказује се alert → модал се затвара → родитељска компонента `GuestsPage` освежава листу преко `onCreated()`.

Позадински сервис у `reservations.ts`

```
export async function saveGuest(payload: SaveGuestRequest): Promise<void> {  
  // POST zahtev za dodavanje novog gosta u bazu  
  // Payload sadrži sve potrebne podatke (ime, jmbg, grad itd.)  
  await http<void>('/guests/save', {  
    method: 'POST',  
    body: JSON.stringify(payload),  
  })  
}
```

СК4 – Измена госта

Улазна тачка — страница `GuestsPage.tsx`

У табели са гостима, сваки ред има дугме „**Info**“ које отвара модални прозор за преглед и уређивање података о том госту:

```
{/* Dugme otvara modal sa detaljima gosta */}  
<button  
  className="btn btn-sm btn-outline-primary"  
  onClick={() => setOpenId(g.id)}  
  style={{ minWidth: 88 }}  
>  
  Info  
</button>
```

Када корисник кликне → у state-у се поставља `openId = g.id`.

Испод у JSX-у се условно приказује:

```
{/* GuestInfoModal prikazuje informacije o gostu.  
   onUpdated → automatski osvežava listu ako je gost izmenjen. */}  
<GuestInfoModal  
  id={openId}  
  show={openId !== null}  
  onClose={() => setOpenId(null)}  
  onUpdated={() => refreshList(query)}  
>
```

id – ID госта који треба учитати.

show – управља видљивошћу модала.

onUpdated() – након измене, освежава листу гостију.

onClose() – затвара модал.

Ово је главни модални прозор који управља и прегледом и изменом података.

Главне state променљиве:

```
// loading označava da li trenutno traje poziv ka serveru (učitavanje ili snimanje)
const [loading, setLoading] = useState(false)
// error se koristi da se prikaže poruka o grešci u alertu unutar modala
const [error, setError] = useState<string | null>(null)
// guest sadrži podatke o gostu koje dobijamo iz API-ja
const [guest, setGuest] = useState<Guest | null>(null)
// cities sadrži listu svih gradova, potrebnu za dropdown u formi
const [cities, setCities] = useState<City[]>([])
// edit označava da li su polja forme omogućena za izmenu (switch dugme na dnu)
const [edit, setEdit] = useState(false)

// form čuva kopiju podataka gosta koji se prikazuju u inputima.
// U ovoj kopiji se prave izmene dok se ne klikne "Sačuvaj".
const [form, setForm] = useState<UpdateGuestRequest | null>(null)
```

Учитавање података о госту при отварању модала

useEffect() се активира кад се модал отвори (и постоји ID):

```
// Kada se modal otvori (show === true) i postoji id,
// učitavaju se istovremeno detalji gosta i lista svih gradova.
useEffect(() => {
  if (!show || id == null) return
  setEdit(false)
  setLoading(true)
  setError(null)

  ;(async () => {
    try {
      // Promise.all omogućava paralelno učitavanje podataka gosta i gradova
      const [g, cs] = await Promise.all([findGuestById(id), listCities()])

      // Debug ispis (može pomoći prilikom razvoja)
      console.log('Guest data:', g)
      console.log('Guest cityId:', g.city.cityId)
      console.log('Available cities:', cs)

      // Nakon uspešnog poziva, ažuriramo state
      setGuest(g)
      setCities(cs)

      // Inicijalizujemo formu trenutnim podacima gosta
      setForm({
        id: g.id,
        jmbg: g.jmbg,
        firstName: g.firstName,
        lastName: g.lastName,
        email: g.email,
        phoneNumber: g.phoneNumber,
        cityId: g.city.cityId ?? null, // ako nema grada, postavljamo null
      })
    } catch (e: any) {
      setError(e.message ?? 'Neuspelo učitavanje detalja')
    } finally {
      setLoading(false)
    }
  })()
}, [show, id])
```

Када се модал отвори, истовремено се шаљу два API захтева:

findGuestById(id) → добија податке о конкретном госту.

listCities() → пуни листу градова за падајући мени.

Добијени подаци се смештају у state guest, cities и form.

edit се ресетује на false → иницијално је форма у режиму прегледа.

Приказ форме и укључивање режима „уређивања“

Сва input поља у JSX-у добијају проп disabled={!edit}.

На дну форме постоји прекидач или дугме које укључује режим измене:

```
/* Switch omogućava uključivanje režima uređivanja */
<div className="form-check form-switch">
  <input
    className="form-check-input"
    type="checkbox"
    id="editSwitch"
    checked={edit}
    onChange={(e) => setEdit(e.target.checked)}
  />
  <label className="form-check-label" htmlFor="editSwitch">
    Uredi
  </label>
</div>
```

када је edit = true, сви input-и постају активни.

Ажурирање поља форме

Функција onChange() ажурира само одређено поље у form state-у:

```
// onChange ažurira lokalni state forme prilikom svake promene inputa
const onChange = (patch: Partial<UpdateGuestRequest>) => {
  setForm((f) => ({ ...(f as UpdateGuestRequest), ...(patch as any) })))
}
```


Пример:

```
{/* Polje Email */}
<div className="col-sm-6">
  <label className="form-label">Email</label>
  <input
    type="email"
    className="form-control"
    value={form.email}
    onChange={(e) => onChange({ email: e.target.value })}
    disabled={!edit}
  />
</div>
```

Слање измењених података — функција submit()

```
// Funkcija koja šalje izmenjene podatke serveru
const submit = async () => {
  if (!form) return
  if (!validateForm()) return

  try {
    setLoading(true)
    await updateGuest(form) // PUT /guest/update
    setEdit(false)
    onUpdated() // obaveštava roditelja da osveži listu
    onClose()
    alert('Podaci o gostu su uspešno ažurirani!')
  } catch (e: any) {
    setError(e?.message ?? 'Ažuriranje nije uspelo')
  } finally {
    setLoading(false)
  }
}
```

Врши се валидација.

Позива се updateGuest(form) из reservations.ts.

Backend чува измене у бази.

Након успеха → alert, модал се затвара, родитељ (GuestsPage) се освежава преко onUpdated().

Сервисни позив у reservations.ts

```
export async function updateGuest(payload: UpdateGuestRequest): Promise<void> {  
  // PUT zahtev za ažuriranje već postojećeg gosta  
  // PUT se koristi kada menjamo postojeći entitet (za razliku od POST koji pravi novi)  
  await http<void>('/guests/update', {  
    method: 'PUT',  
    body: JSON.stringify(payload), // Pretvaramo objekat u JSON  
  })  
}
```

JSX изглед:

```
{/* Dugme za snimanje izmena - aktivno samo ako je edit uključen */}  
<button  
  type="button"  
  className="btn btn-primary"  
  onClick={submit}  
  disabled={!edit || loading || !form}  
>  
  Sačuvaj izmene  
</button>
```

CK5 – Брисање госта

На крају сваког реда у табели гостију постоји дугме „Обриши“.
Оно активира функцију onDelete(id) која управља целим процесом брисања:

```
</button>

{/* Dugme pokreće brisanje gosta */}
<button
  className="btn btn-sm btn-outline-danger"
  onClick={() => onDelete(g.id)}
  style={{ minWidth: 88 }}
>
  Obriši
</button>
```

Главна функција — onDelete(id)

```
// Traži potvrdu korisnika, poziva backend, a zatim ažurira lokalni state.
const onDelete = async (id: string | number) => {
  const yes = window.confirm(
    'Da li ste sigurni da želite da obrišete ovog gosta? (brisanje neće biti moguće ako postoje rezervacije vezane za njega)'
  )
  if (!yes) return
  try {
    setDeletingId(id) // koristi se za disable stanja ako bi se naknadno dodalo
    await deleteGuestById(id) // HTTP DELETE /guests/deleteById?id=...
    // Lokalno uklanjamo obrisanih gosta iz prikazane liste bez ponovnog API poziva
    setData((prev) => prev.filter((r) => String(r.id) !== String(id)))
    alert('Gost je uspešno obrisani!')
  } catch (e: any) {
    alert(e?.message ?? 'Brisanje nije uspelo')
  } finally {
    setDeletingId(null)
  }
}
```

Корисник кликне „Обриши“ → позива се onDelete(id).

Приказује се **потврда (confirm дијалог)** да корисник потврди акцију.

Ако изабере „Не“ → функција се завршава.

Ако потврди → поставља се deletingId = id (да се дугме привремено онемогући),

позива се deleteGuestById(id) → брише се гост на backend-у.

Ако backend врати успех:

локално се избацује тај ред из data без поновног позива refreshList().

приказује се порука „Гост је успешно обрисан!“

Ако дође до грешке (нпр. постоје везане резервације) → alert са поруком „Брисање није успело“.

На крају, deletingId се враћа на null → дугме се поново активира.

Сервисни позив — **reservations.ts**

```
export async function deleteGuestById(id: number | string): Promise<void> {  
  // Briše gosta iz baze  
  // DELETE zahtev sa query parametrom id  
  await http<void>(`/guests/deleteById?id=${id}`, { method: 'DELETE' })  
}
```

Обрада на интерфејсу

Када се setData() позове:

```
// Lokalno uklanjamo obrisanog gosta iz prikazane liste bez ponovnog API poziva  
setData((prev) => prev.filter((r) => String(r.id) !== String(id)))
```

React одмах поново рендерује табелу и уклања тај ред — **без потребе за новим учитавањем података.**

Контрола стања дугмета током брисања

```
{/* Dugme pokreće brisanje gosta */}  
<button  
  className="btn btn-sm btn-outline-danger"  
  onClick={() => onDelete(g.id)}  
  style={{ minWidth: 88 }}  
>  
  Obriši  
</button>
```

док се одговарајући гост брише, његово дугме је онемогућено.
→ спречава се више кликова и дупли DELETE захтеви.

СК6 – Претрага резервације

Улазна тачка — страница ReservationsPage.tsx

У горњем делу странице налази се **форма за претрагу** резервација. Она садржи уносно поље и дугме које активира претрагу преко функције `onSearchSubmit`.

```
/* Forma za pretragu rezervacija */  
<form className="d-flex" onSubmit={onSearchSubmit} role="search">  
  <input  
    className="form-control me-2"  
    style={{ width: 260 }}  
    type="search"  
    placeholder="Pretraga (gost, soba, ID...)"  
    value={query}  
    onChange={(e) => setQuery(e.target.value)} // ažurira stanje – controlled komponenta  
  />  
  <button className="btn btn-outline-secondary" type="submit" disabled={searching}>  
    {searching ? 'Tražim...' : 'Traži'}  
  </button>  
</form>
```

Обрада submit догађаја форме

```
// Obrada submit događaja forme – klasičan React pattern:  
// preventDefault sprečava reload, pozivamo custom funkciju za pretragu.  
const onSearchSubmit = (e: React.FormEvent) => {  
  e.preventDefault()  
  runSearch(query)  
}
```

Главна функција — runSearch(q)

Функција обрађује логички ток претраге.

Проверава да ли је унос празан и у зависности од тога позива различите API методе.

```
// Conditional logic: ako je unos prazan → prikazuje sve; inače koristi backend search endpoint.
// Ovo pokazuje *conditional async flow* – isti pattern koji se koristi u dashboard filtrima.
const runSearch = async (q: string) => {
  const term = q.trim()
  if (!term) {
    setSearching(true)
    try {
      const all = await listReservations()
      setData(all)
    } finally {
      setSearching(false)
    }
    return
  }

  setSearching(true)
  try {
    const res = await searchReservations(term)
    setData(res)
  } catch (e: any) {
    setError(e?.message ?? 'Pretraga nije uspjela')
  } finally {
    setSearching(false)
  }
}
```

Корисник уноси текст у поље → setQuery() ажурира локални state query.

Клик на дугме „Тражи“ позива onSearchSubmit().

onSearchSubmit() спречава освежавање странице и позива runSearch(query).

Ако је поље празно → позива се listReservations() и приказују се све резервације.

Ако има текста → позива се searchReservations(query) да backend врати само оне које одговарају упиту.

Резултат (низ резервација) се смешта у data → React поново исцртава табелу.

Ако дође до грешке → у state се уписује error, па се приказује alert.

Backend комуникација — searchReservations()

```
export async function searchGuests(query: string): Promise<Guest[]> {  
  // Trimujemo i proveravamo da li postoji upit  
  const q = query?.trim() ?? ''  
  // Ako korisnik nije uneo tekst (query je prazan), vraćamo prazan niz – ne šaljemo zahtev  
  if (!q) return []  
  // Ako upit postoji, formira se URL sa query parametrom  
  return http<Guest[]>(`/guests/search?query=${encodeURIComponent(q)}`)  
}
```

Шаље **GET** захтев ка /reservations/search?query=....

Backend враћа све резервације које садрже задати текст (по госту, соби, ID-ју итд.).

Приказ резултата у табели

```
<table className="table table-hover align-middle mb-0">  
  <thead className="table-light">  
    <tr>  
      <th style={{ width: 160 }}>Šifra rezervacije</th>  
      <th>Soba</th>  
      <th>Datum od</th>  
      <th>Datum do</th>  
      <th>Gost</th>  
      <th style={{ width: 100 }} className="text-end">Akcije</th>  
    </tr>  
  </thead>  
  <tbody>  
    { /* Ako nema rezultata – prikazuje se fallback poruka */ }  
    { data.length === 0 && (  
      <tr>  
        <td colspan={6} className="text-center text-muted p-4">  
          Nema rezervacija.  
        </td>  
      </tr>  
    ) }  
  </tbody>  
</table>
```

CK7 – Kreiranje rezervacije

Otvaranje modala — korisnik klikne „Nova rezervacija“. State createOpen prelazi na true. JSX uslovno renderuje modal i predaje onClose i onCreated.

```
/* Dugme koje otvara modal za novu rezervaciju */  
<button className="btn btn-primary ms-2" onClick={() => setCreateOpen(true)}>  
  Nova rezervacija  
</button>  
</div>
```

```
/* NewReservationModal – modal za kreiranje nove rezervacije.  
   Prosleđuje callback onCreated koji poziva refresh roditelja (lifting state up). */  
<NewReservationModal  
  show={createOpen}  
  onClose={() => setCreateOpen(false)}  
  onCreated={refresh}  
>
```

Korak aktivira child komponentu i spaja tok zatvaranja i osvežavanja roditelja.

Reset pri zatvaranju — kada show postane false, modal vraća formu u početno stanje.

```
setGuestQuery(''); setGuestResults([]); setSelectedGuest(null)  
setRoomQuery(''); setRoomResults([]); setSelectedRoom(null)  
setDateFrom(''); setDateTo(''); setBreakfastIncluded(false); setNote('')  
setError(null); setSubmitting(false)
```

Opis: Sprečava curenje prethodnih izbora i poruka pri sledećem otvaranju.

Pretraga gosta — debounce 350 ms. Ispod 2 slova nema poziva. Greške se hvataju u lokalno stanje.

```
// Ideja: ne gađaj server na svako slovo. Sačekaj 350 ms nakon poslednjeg unosa pa pozovi API.  
// Ako korisnik nastavi da kuca pre isteka 350 ms → prethodni timeout se čisti (clearTimeout).  
useEffect(() => {  
  if (!show) return // nema pretrage dok modal nije otvoren  
  const id = setTimeout(async () => {  
    // Minimalna dužina upita za gosta: 2, da se izbegnu suviše široke pretrage.  
    if (guestQuery.trim().length < 2) { setGuestResults([]); return }  
    try {  
      // Koristimo trenutnu vrednost guestQuery iz zatvorenog scope-a  
      // jer je to snapshot u času kreiranja timeout-a (debounce pattern).  
      setGuestResults(await searchGuests(guestQuery))  
    } catch (e: any) {  
      // Grešku prijavljujemo korisniku; ne bacamo dalje jer je UX važniji.  
      setError(e?.message ?? 'Greška pri pretrazi gostiju')  
    }  
  }, 350)  
  return () => clearTimeout(id) // ključ debouncing-a: otkaži prethodni tajmer ako se query brzo  
}, [guestQuery, show])
```


Pretraga sobe — isti obrazac, ali minimalno 1 znak jer broj sobe može biti jednocifren.

```
// Isti logika kao za goste. Minimalna dužina 1 je ok jer broj sobe može biti 1 cifra
useEffect(() => {
  if (!show) return
  const id = setTimeout(async () => {
    if (roomQuery.trim().length < 1) { setRoomResults([]); return }
    try {
      setRoomResults(await searchRooms(roomQuery))
    } catch (e: any) {
      setError(e?.message ?? 'Greška pri pretrazi soba')
    }
  }, 350)
  return () => clearTimeout(id)
}, [roomQuery, show])
```

Simetrično rešenje za sobe, isti debounce i handling greške.

Datumi → broj noćenja — razlika u danima sa zaštitom od negativnog opsega i problema sa TZ.

```
// useMemo sprečava nepotrebna računanja na svaki render.
// Račun: razlika u danima, uz zaštitu od negativnih vrednosti (npr. ako je dateTo pre dateFrom)
const nights = useMemo(() => {
  if (!dateFrom || !dateTo) return 0
  // Dodajemo T00:00:00 da eliminisemo probleme sa vremenskom zonom pri new Date('YYYY-MM-DD').
  const start = new Date(dateFrom + 'T00:00:00')
  const end = new Date(dateTo + 'T00:00:00')
  const diff = (end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)
  // Math.floor: 2.9 dana je 2 noći. Negativno → 0 (nevalidan opseg).
  return diff > 0 ? Math.floor(diff) : 0
}, [dateFrom, dateTo])
```

Računa validan broj noćenja za dalje cene i validaciju.

Cena i total — uzima se pricePerNight iz tipa sobe; total je memorisan.

```
const pricePerNight = selectedRoom?.roomType?.pricePerNight ?? 0

// ----- OBRAČUN UKUPNE CENE -----
// Ako je pricePerNight string "120", nights * pricePerNight u JS daje broj 120 * 3 = 360.
// Ako je npr. "120 EUR", rezultat bi bio NaN — UI će to videti kroz fmtPrice.
const totalPrice = useMemo(() => (nights * pricePerNight), [nights, pricePerNight])
```

Sprečava re-izračun na svaki render i pokriva slučaj neizabrane sobe.

Validacija i dugme — forma je spremna tek kad su entiteti i datumi validni.

```
// Mora postojati izabran gost i soba, oba datuma, i nights > 0.
// Ovim izbegavamo slanje nepotpunih podataka i nepotreban round-trip do servera.
const canSubmit = !!(selectedGuest && selectedRoom && dateFrom && dateTo && nights > 0)
```

Blokira submit i render dugmeta dok forma nije kompletna.

```
<button type="button" className="btn btn-primary" onClick={submit} disabled={!canSubmit || submitting}>
  {submitting ? 'Čuvam...' : 'Sačuvaj'}
</button>
```

Payload — sklapanje DTO-a sa employeeId, guestId, roomId.

```
// Uzimamo ID trenutno prijavljenog korisnika (zaposlenog) iz localStorage-a.
// Ovaj ID backend koristi kao "employeeId" koji je kreirao rezervaciju.
const employeeIdValue = getUserId()

// SelectedGuest i selectedRoom su ovde definisani jer canSubmit to garantuje.
// TypeScript non-null assertion (!) je bezbedan u ovom grananju.
const payload: SaveReservationRequest = {
  dateFrom,
  dateTo,
  breakfastIncluded,
  note,
  employeeId: employeeIdValue,
  guestId: selectedGuest!.id,
  roomId: selectedRoom!.roomId,
}
```

Svi ID-jevi su eksplicitni i spremni za POST.

Slanje — POST, poruka, zatvaranje i signal roditelju za refresh; greške ostaju vidljive.

```
const submit = async () => {
  if (!canSubmit) return // dodatna zaštita, iako je dugme disable-ovano
  setSubmitting(true)
  setError(null)

  // Uzimamo ID trenutno prijavljenog korisnika (zaposlenog) iz localStorage-a.
  // Ovaj ID backend koristi kao "employeeId" koji je kreirao rezervaciju.
  const employeeIdValue = getUserId()

  // SelectedGuest i selectedRoom su ovde definisani jer canSubmit to garantuje.
  // TypeScript non-null assertion (!) je bezbedan u ovom grananju.
  const payload: SaveReservationRequest = {
    dateFrom,
    dateTo,
    breakfastIncluded,
    note,
    employeeId: employeeIdValue,
    guestId: selectedGuest!.id,
    roomId: selectedRoom!.roomId,
  }

  try {
    await saveReservation(payload) // POST /reservations/save
    alert('Rezervacija je sačuvana!')
    onClose() // zatvori modal odmah po uspehu
    onCreated() // signal roditelju da re-fetchuje listu
  } catch (e: any) {
    // Prikazujemo korisničku poruku. Ne gasimo modal automatski da korisnik vidi poruku
    setError(e?.message ?? 'Snimanje nije uspelo')
  } finally {
    setSubmitting(false) // uvek otključaj dugme
  }
}
```

Centralni akcioni korak. Zaključava UI, poziva API i orkestrira UI posle odgovora.

Osvežavanje liste — roditelj povlači nove podatke i ažurira tabelu.

```
// Koristi se nakon što se napravi nova rezervacija ili obriše postojeća.  
// Demonstrira "state lifting" – child komponenta (NewReservationModal) može izazvati refresh roditelja.  
const refresh = async () => {  
  setLoading(true)  
  setError(null)  
  try {  
    setData(await listReservations())  
  } catch (e: any) {  
    setError(e?.message ?? 'Neuspešno učitavanje rezervacija')  
  } finally {  
    setLoading(false)  
  }  
}
```

Jedinstvena tačka istine za tabelu. Poziva se i posle kreiranja i posle brisanja.

CK8 – Брисање резервације

Dugme u tabeli pokreće akciju brisanja. Klik poziva

```
{/* Dugme za brisanje rezervacije – koristi optimistic update */}
<button
  className="btn btn-sm btn-outline-danger"
  onClick={() => onDelete(r.id)}
  style={{ minWidth: 88 }}
>
  Obriši
</button>
```

Handler onDelete traži potvrdu, setuje ID, zove servis deleteReservationById(id), pa optimistički uklanja red iz state-a. Na uspeh daje poruku, na grešku ispisuje alert, u finally čisti ID.

```
// Demonstrira *optimistic UI update* – ažurira se prikaz odmah po uspešnom DELETE zahtevu.
const onDelete = async (id: string | number) => {
  const yes = window.confirm('Da li ste sigurni da želite da obrišete ovu rezervaciju?')
  if (!yes) return
  try {
    setDeletingId(id)
    await deleteReservationById(id) // DELETE /reservations/deleteById?id=...
    // Lokalno uklanjanje bez novog API poziva – performance-friendly pristup
    setData((prev) => prev.filter((r) => String(r.id) !== String(id)))
    alert('Rezervacija je uspešno obrisana!')
  } catch (e: any) {
    alert(e?.message ?? 'Brisanje nije uspelo')
  } finally {
    setDeletingId(null)
  }
}
```

Servisni sloj vrši stvarni HTTP DELETE prema backendu na ruti
/reservations/deleteById?id=...

```
export async function deleteReservationById(id: number | string): Promise<void> {
  // DELETE zahtev ka endpointu /reservations/deleteById
  // Metoda 'DELETE' govori backendu da treba da obriše resurs sa tim ID-jem
  await http<void>(`/reservations/deleteById?id=${id}`, { method: 'DELETE' })
}
```

State deletingId se koristi za UI signalizaciju da je konkretan red u procesu brisanja (npr. disable dugmeta ili spinner pored reda).

```
// ID rezervacije koja se trenutno briše
const [deletingId, setDeletingId] = useState<string | number | null>(null)
```

Posle uspešnog DELETE poziva refresh() koji re-fetchuje listu sa servera.

```
// Koristi se nakon što se napravi nova rezervacija ili obriše postojeća.  
// Demonstrira "state lifting" – child komponenta (NewReservationModal) može izazvati refresh roditelja.  
const refresh = async () => {  
  setLoading(true)  
  setError(null)  
  try {  
    setData(await listReservations())  
  } catch (e: any) {  
    setError(e?.message ?? 'Neuspešno učitavanje rezervacija')  
  } finally {  
    setLoading(false)  
  }  
}
```

CK9 – Pretraga soba

Forma za pretragu i submit — controlled input ažurira query. Submit poziva `onSearchSubmit(query)`.

```
<form className="d-flex ms-auto" onSubmit={onSearchSubmit} role="search">
  <input
    className="form-control me-2"
    style={{ width: 260 }}
    type="search"
    placeholder="Pretraga (broj sobe, cena)"
    value={query} // kontrolisana vrednost inputa iz state-a
    onChange={(e) => setQuery(e.target.value)} // svaki unos u polje ažurira state
  />
  <button className="btn btn-outline-secondary" type="submit" disabled={searching}>
    /* Dinamički tekst dugmeta u zavisnosti od stanja pretrage */
    {searching ? 'Tražim...' : 'Traži'}
  </button>
</form>
```

```
// React koristi synthetic events. preventDefault() sprečava reload stranice (default HTML ponovno učitavanje)
const onSearchSubmit = (e: React.FormEvent) => {
  e.preventDefault() // zadržava stranicu i sprečava browser da pošalje formu klasičnim putem
  runSearch(query) // pokreće pretragu sa trenutnim unosom iz state-a
}
```

Pokretanje pretrage — `runSearch` pali indikator, zove servis `searchRooms(q)`, ažurira tabelu i gasi indikator. Greške idu u `error`.

```
// Korisnik unosi tekst (query), i na submit se poziva ova funkcija.
// Ako API pretraga ne uspe, postavlja se greška u state-u.
const runSearch = async (q: string) => {
  setSearching(true) // prikazuje loading poruku dok traje pretraga
  try {
    // searchRooms poziva endpoint: /rooms/search?query=...
    const results = await searchRooms(q)
    setData(results) // uspešan odgovor → ažurira tabelu bez reloadovanja stranice
  } catch (e: any) {
    setError(e?.message ?? 'Pretraga nije uspeła')
  } finally {
    setSearching(false)
  }
}
```

Servisni poziv — trimuje upit. Ako je prazan, ne šalje zahtev. Inače gađa `/rooms/search?query=...`

```
export async function searchRooms(query: string): Promise<Room[]> {
  // Trimujemo upit, ako nema teksta ne šaljem zahtev
  const q = query?.trim() ?? ''
  if (!q) return []
  // Vraćamo rezultat GET zahteva sa query parametrom
  return http<Room[]>(`/rooms/search?query=${encodeURIComponent(q)}`)
}
```

Inicijalno učitavanje svih soba — `loadAll()` na mount. Ovo daje početno stanje pre prve pretrage.

```
// Ova funkcija koristi asinhroni API poziv. Koristi try/catch/finally blok radi upravljanja greškama
const loadAll = async () => {
  setLoading(true) // postavlja loading stanje → prikazuje spinner/info poruku
  setError(null) // resetuje eventualnu staru grešku
  try {
    // await čeka da se asinhroni HTTP zahtev završi pre nego što nastavi dalje
    // listRooms() → GET /rooms/findAll
    setData(await listRooms())
  } catch (e: any) {
    // Ako dođe do greške (npr. mrežni problem), postavlja se tekstualna poruka za korisnika
    setError(e?.message ?? 'Neuspešno učitavanje soba')
  } finally {
    // finally blok se uvek izvršava – ovde resetuje loading bez obzira na ishod
    setLoading(false)
  }
}

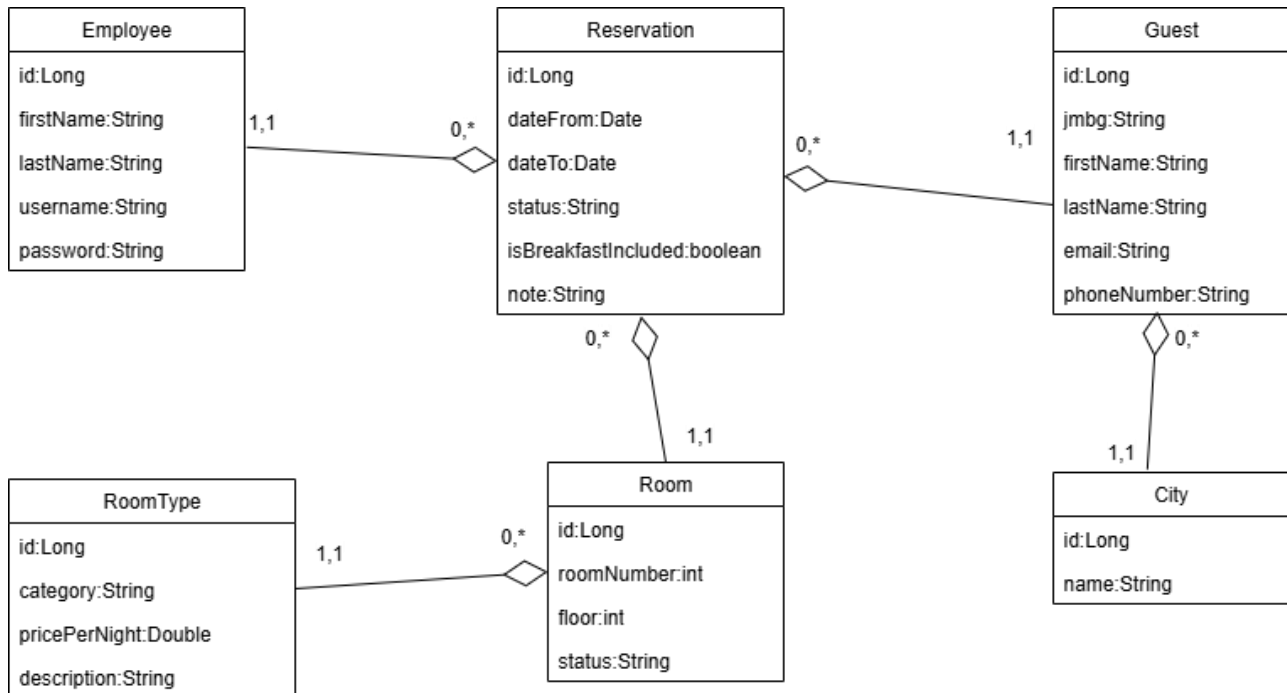
// -----
// useEffect – React lifecycle hook
// -----
// Pokreće se samo jednom (prilikom mountovanja komponente).
// Prazan dependency array [] znači da se efekat ne ponavlja.
useEffect(() => {
  loadAll() // inicijalno učitavanje svih soba po otvaranju stranice
}, [])
```

Tabela koja renderuje data.

```
/* Map funkcija prolazi kroz sve sobe i renderuje redove tabele */
{data.map((r) => (
  <tr key={r.roomId}>
    /* className="text-monospace" koristi monospace font – vizuelno ističe ID */
    <td className="text-monospace">{r.roomId}</td>
    <td>{r.roomNumber}</td>
    <td>{r.floor}</td>
    /* Operator ?. sprečava grešku ako roomType nije definisan */
    <td>{r.roomType?.category}</td>
    <td>{fmtPrice(r.roomType?.pricePerNight)}</td> /* formatira vrednost kao EUR */
  </tr>
))}
```

6. Структура пројекта

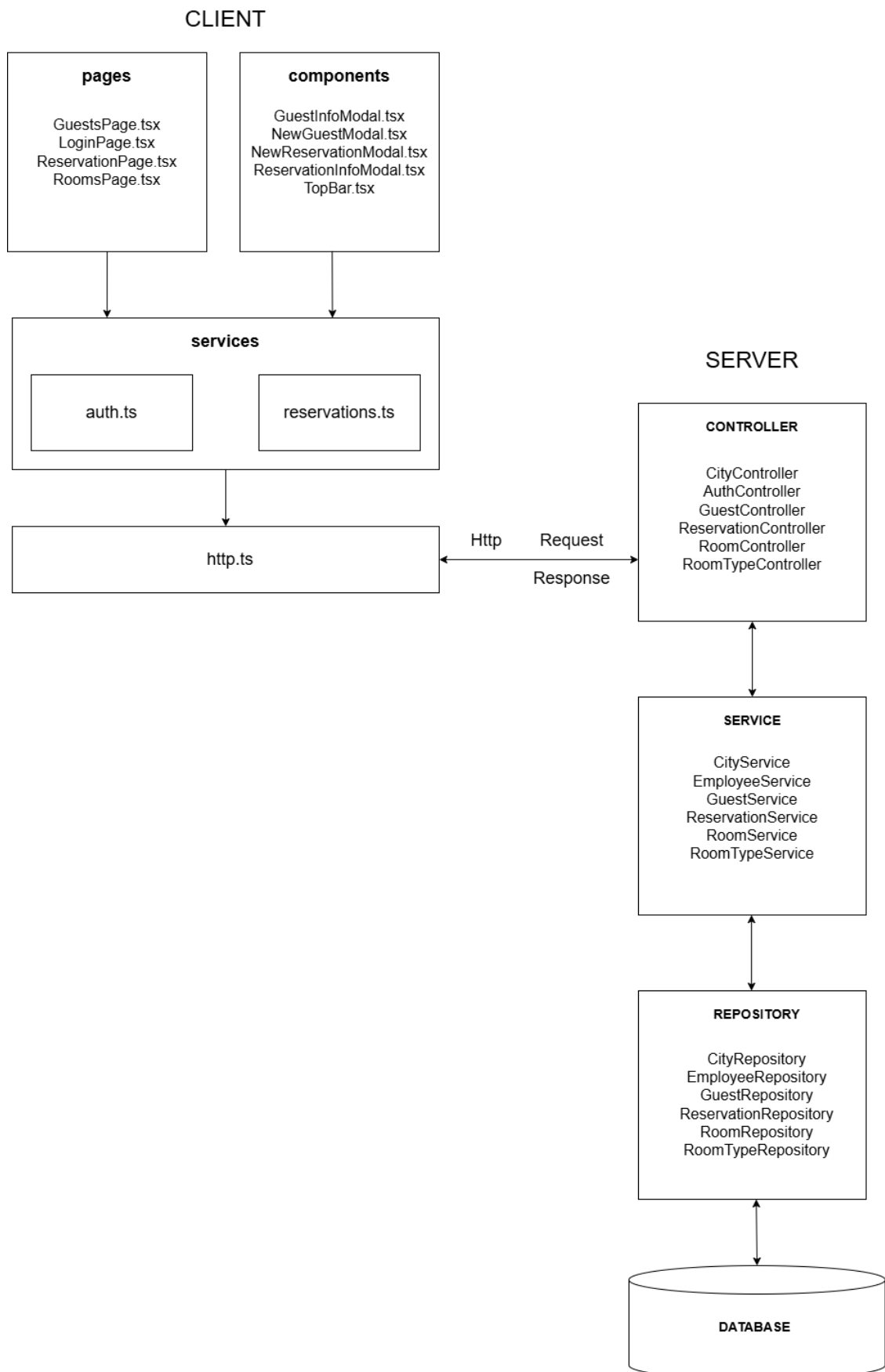
Концептуални модел:



На основу концептуалног модела направљен је следећи релациони модел:

1. **Employee** (idEmployee, firstName, lastName, username, password)
2. **Room** (idRoom, roomNumber, floor, status, *idRoomType*)
3. **RoomType** (idRoomType, category, pricePerNight, description)
4. **City** (idCity, cityName)
5. **Guest** (idGuest, jmbg, firstName, lastName, email, phoneNumber, *idCity*)
6. **Reservation** (idReservation, dateFrom, dateTo, status, isBreakfastIncluded, note, *idEmployee*, *idGuest*, *idRoom*)

7. Архитектура пројекта



8. Закључак

Овај пројекат показује да комбинација **Spring Boot-a** и **React-a** обезбеђује поуздан backend и брз, интерактиван кориснички интерфејс. У поређењу са класичним Java десктоп или конзолним апликацијама, React омогућава динамичке форме, тренутне реакције система и пријатније корисничко искуство. Spring Boot обезбеђује REST API, валидацију података и јасно дефинисане грешке, које се у React делу апликације обрађују помоћу try/catch блока и приказу као поруке или упозорења.

За складиште података коришћен је **MySQL**, са **JPA/Hibernate** мапирањем ентитета (Запослени, Гост, Резервација, Град, Соба, ТипСобе). Ограничења у бази (UNIQUE, FK, CASCADE) осигуравају интегритет података.

Maven је коришћен за управљање зависностима и процесом изградње апликације.

Пројекат представља добру основу за даљи развој — увођење **WebSocket/STOMP** функционалности за реална освежавања, управљање улогама и дозволама, евиденцију активности (audit), **i18n** подршку, интеграциона и е2е тестирања, као и **Docker/CI/CD** аутоматизацију. Могуће је и унапређење календара доступности соба ради боље прегледности и планирања.